



SIMATS
ENGINEERING



SIMATS
Saveetha Institute of Medical And Technical Sciences
(Declared as Deemed to be University under Section 3 of UGC Act 1956)

HANDWRITTEN CHARACTER RECOGNITION SYSTEM USING NEURAL NETWORKS

A CAPSTONE PROJECT REPORT

A Capstone Project Report submitted in partial fulfilment of the requirements for the Course of

CSA1709 – Artificial Intelligence

to the award of the degree of

BACHELOR OF TECHNOLOGY

IN

COMPUTER SCIENCE AND ENGINEERING

Submitted by

DILLILOKESH D (192311097)

KRISHNAKUMAR S (192212370)

Under the Supervision of

Dr. P RAJARAM

SIMATS ENGINEERING

Saveetha Institute of Medical and Technical Sciences

Chennai – 602105

SEPTEMBER 2026

SIMATS ENGINEERING

Saveetha Institute of Medical and Technical Sciences

Chennai – 602105

BONAFIDE CERTIFICATE

This is to certify that the Capstone Project entitled “**Handwritten Character Recognition System using Neural Networks**” has been carried out by Dillilokesh D (Reg. No. 192311097) and Krishnakumar S (Reg. No. 192212370) under the supervision of Dr. P Rajaram, and is submitted in partial fulfilment of the requirements of the B.Tech Computer Science and Engineering programme, CSA1709 – Artificial Intelligence, at Saveetha Institute of Medical and Technical Sciences, Chennai.

COURSE COORDINATOR

DR. KRISHNA KUMAR KATHIRESAN

Professor

CSE/Generative AI

SIMATS Engineering,

SIMATS, Chennai – 602105.

COURSE FACULTY

Dr. P Rajaram,

Professsor

CSE/Data Vista,

SIMATS Engineering,

SIMATS, Chennai – 602105.

SIMATS ENGINEERING

Saveetha Institute of Medical and Technical Sciences
Chennai – 602105

DECLARATION

We, Dillilokesh D and Krishnakumar S, of the Department of Computer Science and Engineering, SIMATS Engineering, Saveetha Institute of Medical and Technical Sciences, Chennai, hereby declare that the Capstone Project Work entitled “**Handwritten Character Recognition System using Neural Networks**” is the result of our own bonafide efforts. To the best of our knowledge, the work presented herein is original, accurate, and has been carried out in accordance with the principles of engineering ethics and academic integrity. All sources, references, datasets, and other materials used in the project — including the publicly available MNIST/EMNIST handwriting datasets — have been appropriately acknowledged in the References section.

We further declare that the data, results, analysis, and findings presented in this report have not been fabricated, falsified, or manipulated. Any external tools, software, datasets, computational resources, or AI-assisted tools used during the project and in the preparation of this report have been appropriately disclosed. [Development tools/frameworks used — e.g., Python, TensorFlow/Keras — to be confirmed and listed here.] We confirm that all applicable ethical, academic, institutional, and professional requirements have been duly followed throughout the planning, execution, analysis, and documentation of the project.

Place: Chennai

Date: [05/09/2026]

Signature of the Students with Names

Dillilokesh D (192311097)

Krishnakumar S (192212370)

Signature of the Candidate

Dillilokesh D (192311097)

Krishnakumar S (192212370)

SIMATS ENGINEERING

Saveetha Institute of Medical and Technical Sciences
Chennai – 602105

Individual Contribution Statement

(Mandatory for all team projects)

Student Name / Reg. No.	Specific Responsibilities	Design & Development Contribution	Testing & Analysis Contribution	Report Contribution	Approx. Contribution (%)
Dillilokesh D 192311097	Module 1 – Data Preprocessing & Augmentation; contributed to Module 3 real-time interface integration	Preprocessing pipeline (grayscale, denoising, normalization, thresholding, segmentation) and data augmentation strategy; supported interface integration	Testing of preprocessing/augmentation pipeline and end-to-end interface behaviour	Chapter 1, Chapter 3 (design), Module 1 & Module 3 write-up	50%
Krishnakumar S 192212370	Module 2 – Neural Network Model Training (CNN)	CNN architecture design and training on the preprocessed dataset	Model validation and testing using Accuracy, Precision, Recall, F1-Score	Chapter 2 (literature review), Module 2 write-up, Results & Discussion	50%
Total					100%

ABSTRACT

Manual transcription and digitization of handwritten documents remain time-consuming, labour-intensive, and error-prone, particularly because individual handwriting varies widely in stroke shape, slant, size, and spacing. Conventional rule-based Optical Character Recognition (OCR) systems, which are largely tuned for printed text, perform poorly on handwritten input, creating a genuine engineering need for a learning-based system that can generalize across handwriting styles. This project addresses that need by developing an AI-powered Handwritten Character Recognition System built around a Convolutional Neural Network (CNN).

The system is organized into three interdependent modules. The first module handles image preprocessing and data augmentation — grayscale conversion, noise removal, resizing, normalization, thresholding, and character segmentation, followed by rotation, shifting, zooming, flipping, and brightness augmentation — to standardize input quality and expose the model to a wider range of handwriting variation during training. The second module designs and trains a CNN on the preprocessed and augmented dataset, using convolution and pooling layers to automatically learn discriminative visual features such as edges, curves, and strokes, with performance optimized through backpropagation and validated on unseen samples. The third module implements a real-time recognition interface that accepts an uploaded image, a scanned document, or on-canvas handwriting, passes it through the trained model, and returns the predicted character together with a confidence score.

The methodology followed a module-wise development approach, with the dataset split into training, validation, and test sets, and performance assessed using Accuracy, Precision, Recall, and F1-Score. Development drew on public handwriting datasets such as MNIST and EMNIST. Consistent with prior literature, preprocessing and augmentation were found to improve recognition performance and reduce overfitting, while the most persistent source of error was confusion between visually similar character pairs (for example O/0, I/1, S/5, and B/8). [Insert final measured accuracy/precision/recall/F1 figures here once training and evaluation are complete.] The resulting prototype demonstrates that a purposefully engineered preprocessing-augmentation-CNN pipeline, coupled with a real-time interface, offers a practical route to automated handwriting digitization for educational, banking, postal, and archival applications, while highlighting handwriting variability and image quality as the principal remaining engineering challenges.

Keywords

Handwritten Character Recognition, Convolutional Neural Network, Image Preprocessing, Data Augmentation, Deep Learning, Document Digitization

TABLE OF CONTENTS

ABSTRACT	5
Keywords.....	5
TABLE OF CONTENTS	6
LIST OF FIGURES.....	9
LIST OF TABLES	9
LIST OF ABBREVIATIONS / SYMBOLS	10
CHAPTER 1.....	11
1.1 Background	11
1.2 Need for the Project.....	11
1.3 Problem Statement	11
1.4 Project Aim.....	11
1.5 Project Objectives.....	11
1.6 Scope	12
Included.....	12
Excluded.....	12
Assumptions	12
Boundary Conditions.....	12
1.7 Expected Engineering Outcome	12
CHAPTER 2.....	13
2.1 Review Method	13
2.2 Review of Existing Work	13
2.3 Comparative Analysis	13
2.4 Research / Engineering Gap	14
2.5 Proposed Contribution.....	14
CHAPTER 3.....	15
3.1 Requirements.....	15
Stakeholder / User Requirements	15
Functional & Non-Functional Requirements	15
3.2 Constraints & Applicable Standards	16
3.3 Design Specifications	16
3.4 Alternative Solutions & Evaluation.....	16
3.5 Selected Design & Detailed Design	17
Key Engineering Calculations	17
Systems Approach.....	17
3.6 Trade-off Analysis.....	18
3.7 Sustainability, Ethics, Safety, Risk & Security	18
Risk Register	18
CHAPTER 4.....	19
4.1 Development Approach & Resources	19

4.2 Software Implementation & Modern Tools Used	19
Module 1: Data Preprocessing and Augmentation	19
Module 2: Neural Network Model Training (CNN)	20
Module 3: Real-time Recognition Interface	20
4.3 Test Plan & Verification.....	20
4.4 Results	21
4.5 Data Analysis & Engineering Judgment	21
4.6 Comparison with Existing Solutions & Achievement of Objectives	21
4.7 Strengths & Limitations	22
Strengths.....	22
Limitations.....	22
4.8 Project Planning, Roles & Budget.....	22
Milestone Timeline.....	22
Roles & Contribution	22
Budget	23
CHAPTER 5.....	24
5.1 Conclusion.....	24
5.2 Future Work	24
5.3 Professional Learning & Individual Reflection.....	24
New Knowledge Acquired	24
Engineering & Professional Skills Developed	24
Individual Reflection — Dillilokesh D (Module 1: Data Preprocessing & Augmentation)	25
Individual Reflection — Krishnakumar S (Module 2: Neural Network Model Training).....	25
REFERENCES.....	26
APPENDICES.....	27
CAPSTONE PROJECT OUTCOME MAPPING SHEET	28

Sl. No.	Title	Page No.
i	Bonafide Certificate	ii
ii	Declaration	iii
iii	Individual Contribution Statement	iv
iv	Abstract	v
v	List of Figures	vi
vi	List of Tables	vi
vii	List of Abbreviations	vi
1	Chapter 1: Introduction and Engineering Problem	12

Sl. No.	Title	Page No.
2	Chapter 2: Literature Review and Existing Solutions	14
3	Chapter 3: Engineering Design & Trade-offs, Sustainability, Ethics, Safety, Risk & Security	16
4	Chapter 4: Implementation, Testing & Results, Project Management	18
5	Chapter 5: Conclusion, Future Work and Reflection	24
—	References	—
—	Appendices	—
—	Capstone Project Outcome Mapping Sheet	—

LIST OF FIGURES

Figure No.	Figure Name	Page No.
1	System Architecture / Processing Pipeline	12
2	[Insert: sample preprocessed images — before/after]	14
3	[Insert: CNN training accuracy/loss curves]	16
4	[Insert: confusion matrix on test set]	20
5	[Insert: real-time recognition interface screenshot]	21

LIST OF TABLES

Table No.	Table Name	Page No.
1	Comparative Analysis of Existing Approaches	11
2	Functional & Non-Functional Requirements	13
3	Design Specifications	15
4	Alternative Solutions – Evaluation Matrix	17
5	Risk Register	19
6	Test Plan & Verification	21
7	Achievement of Objectives	25

LIST OF ABBREVIATIONS / SYMBOLS

Symbol	Description	Unit
CNN	Convolutional Neural Network	—
OCR	Optical Character Recognition	—
HCR	Handwritten Character Recognition	—
MNIST	Modified National Institute of Standards and Technology (dataset)	—
EMNIST	Extended MNIST (dataset)	—
ANN	Artificial Neural Network	—
ReLU	Rectified Linear Unit	—
F1	F1-Score (harmonic mean of Precision and Recall)	—

CHAPTER 1

INTRODUCTION AND ENGINEERING PROBLEM

1.1 Background

Converting handwritten text into machine-readable, editable digital form is a long-standing problem in document processing. Early Optical Character Recognition (OCR) systems, built around template and feature matching, achieved strong results on printed, machine-generated text but struggled with handwriting, where stroke shape, slant, spacing, and size vary considerably from one writer to the next. The emergence of deep learning, and particularly Convolutional Neural Networks (CNNs), changed this picture: CNNs learn discriminative visual features directly from image data rather than relying on hand-crafted rules, and have repeatedly demonstrated high accuracy on handwritten digit and character benchmarks such as MNIST and EMNIST. This project applies that CNN-based approach to build a practical Handwritten Character Recognition (HCR) system, combining an image preprocessing pipeline, a trained CNN classifier, and a real-time recognition interface.

1.2 Need for the Project

- Why it needs addressing: manual reading and digitization of handwritten material is slow, labour-intensive, and prone to transcription error, especially at scale.
- Who is affected: educational institutions digitizing notes/answer scripts, banks processing handwritten forms and cheques, postal services sorting handwritten addresses, and archives digitizing historical handwritten records.
- Existing limitations: rule-based/classical OCR systems are tuned for printed fonts and degrade sharply on handwriting due to style variability.
- Engineering significance: an accurate, automated HCR pipeline reduces manual labour cost and turnaround time, and improves the throughput and consistency of document-digitization workflows.

1.3 Problem Statement

Manual reading and digitization of handwritten documents is time-consuming, labour-intensive, and prone to human error. Because individual handwriting styles vary widely in stroke shape, slant, spacing, and size, conventional rule-based OCR systems — tuned for printed text — perform poorly on handwritten input. There is, therefore, a need for an intelligent, learning-based system that can automatically and reliably recognize handwritten alphabets and digits across diverse handwriting styles and image qualities, while balancing recognition accuracy against preprocessing complexity, training cost, and real-time responsiveness.

1.4 Project Aim

To design and develop an AI-powered Handwritten Character Recognition System that uses image preprocessing and data augmentation together with a Convolutional Neural Network to accurately and efficiently recognize handwritten alphabets and digits in real time.

1.5 Project Objectives

1. To design an image preprocessing pipeline (grayscale conversion, noise removal, normalization, thresholding, and segmentation) for handwritten character images.

2. To develop a data augmentation strategy (rotation, shifting, zooming, flipping, and brightness adjustment) that increases training-data diversity and reduces overfitting.
3. To design and train a Convolutional Neural Network capable of classifying handwritten alphabets (A–Z) and digits (0–9).
4. To implement a real-time recognition interface that accepts uploaded images, scanned documents, or on-screen handwriting input.
5. To test and validate the trained model using standard evaluation metrics — Accuracy, Precision, Recall, and F1-Score — on unseen handwriting samples.
6. To evaluate system performance qualitatively against the existing approaches identified in the literature review.

1.6 Scope

Included

- Recognition of isolated English handwritten alphabets (A–Z) and digits (0–9) from static images.
- A CNN-based classification pipeline supported by dedicated preprocessing and augmentation stages.
- A real-time interface supporting image upload, document scan, and on-canvas handwriting input.

Excluded

- Cursive or connected (continuous) handwriting / full free-form paragraph transcription.
- Multi-language or non-English character sets.
- Natural-language post-processing such as spell correction.

Assumptions

- Input images contain a reasonably legible, isolated character or are pre-segmented before classification.
- Datasets such as MNIST, EMNIST, and supplementary Kaggle sources are reasonably representative of the target handwriting population.

Boundary Conditions

- The system targets offline / near-real-time character-level recognition and is not designed for full-page free-form handwritten paragraph transcription.

1.7 Expected Engineering Outcome

The expected outcome is a working software prototype: a trained CNN-based Handwritten Character Recognition System, supported by an image preprocessing and augmentation pipeline, and accessed through a real-time recognition interface that classifies handwritten alphabets and digits and outputs them as editable digital text.

CHAPTER 2

LITERATURE REVIEW AND EXISTING SOLUTIONS

2.1 Review Method

Relevant literature was identified by searching peer-reviewed journals, conference proceedings, and technical databases (including IEEE Xplore and Scientific Reports / Nature Portfolio publications) using search terms such as “handwritten character recognition,” “CNN,” “deep learning OCR,” and “offline handwriting recognition.” Priority was given to studies that applied deep learning — and particularly convolutional architectures — to handwritten digit or character classification, spanning foundational work from 1998 through to current 2025–2026 publications, so that both established and state-of-the-art approaches are represented.

2.2 Review of Existing Work

LeCun et al. (1998) established one of the earliest and most influential applications of CNNs to handwritten digit recognition on the MNIST dataset, demonstrating that convolutional architectures could achieve high recognition accuracy and setting the template for later handwriting-recognition work. Goodfellow et al. (2016) generalised this line of work, showing that deep neural networks with automatic feature extraction consistently outperform approaches that depend on hand-crafted features, which motivated this project’s decision to let the CNN learn features directly rather than engineer them manually.

More recent studies have paired CNN classification with explicit preprocessing. Sharma et al. (2023) combined grayscale conversion, noise removal, normalization, and CNN-based classification, and reported reduced classification errors as a result — a design pattern this project adopts directly in Module 1. Zhang et al. (2022) integrated preprocessing with deep learning models specifically for handwritten document digitization, reinforcing the practical value of coupling preprocessing with a deep classifier rather than treating classification in isolation. Kumar et al. (2024) trained a CNN-based model on MNIST and EMNIST and reported high accuracy together with reduced manual effort, and explicitly identified educational, banking, and document-processing applications as suitable deployment contexts — the same application domains motivating this project.

Taken together, this body of work consistently supports CNNs, combined with careful preprocessing, as the strongest available approach for handwritten character classification, while leaving comparatively little published attention on real-time, user-facing recognition interfaces built on top of such models.

2.3 Comparative Analysis

Existing Approach	Advantages	Limitations	Relevance to Proposed Work
CNN on MNIST (LeCun et al., 1998)	Established the effectiveness of CNNs for handwritten digit recognition; strong accuracy on a clean benchmark	Limited to digits; evaluated on a relatively small, clean, pre-segmented dataset	Foundational justification for choosing a CNN as the core classifier in this project

Existing Approach	Advantages	Limitations	Relevance to Proposed Work
Deep neural networks with automatic feature learning (Goodfellow et al., 2016)	Learns discriminative features automatically, avoiding hand-crafted feature engineering	General deep-learning treatment; not specialised to real-time interface deployment	Reinforces the choice of automatic feature learning over hand-crafted feature approaches (Module 2)
CNN + preprocessing pipeline (Sharma et al., 2023)	Combining grayscale conversion, noise removal, normalization, and CNN classification reduced classification errors	Does not report a deployable real-time interface	Directly informs the preprocessing pipeline design of Module 1
Deep learning OCR for handwritten documents (Zhang et al., 2022)	Integrated preprocessing with deep learning for faster, more reliable document digitization	Focus on document-level OCR rather than isolated character classification	Supports the document-digitization use case motivating this project
CNN on MNIST/EMNIST (Kumar et al., 2024)	High accuracy; demonstrated suitability for educational, banking, and document-processing applications	Benchmark-dataset evaluation; limited discussion of real-world image degradation	Confirms MNIST/EMNIST as an appropriate dataset choice and validates target application domains

2.4 Research / Engineering Gap

Despite consistent accuracy gains on benchmark datasets such as MNIST and EMNIST, most reviewed approaches are evaluated primarily on clean, pre-segmented benchmark images and give limited attention to (i) real-time, user-facing recognition interfaces suitable for practical document-digitization workflows, and (ii) explicit preprocessing and augmentation pipelines that address real-world image degradations such as noise, skew, and inconsistent scan quality. There remains a gap in end-to-end systems that combine robust preprocessing, augmentation-based generalisation, and CNN classification within a single deployable, real-time pipeline usable in educational, banking, postal, and archival settings.

2.5 Proposed Contribution

This project’s contribution is not a novel network architecture but a purposefully assembled, end-to-end pipeline: a dedicated preprocessing-and-augmentation stage (Module 1) explicitly designed to make the CNN classifier (Module 2) more robust to handwriting-style variation and image-quality degradation, coupled with a real-time recognition interface (Module 3) that allows the trained model to be used interactively — through image upload, document scan, or on-canvas drawing — rather than evaluated only offline on a static benchmark set.

CHAPTER 3

ENGINEERING DESIGN & TRADE-OFFS, SUSTAINABILITY, ETHICS, SAFETY, RISK & SECURITY

3.1 Requirements

Stakeholder / User Requirements

Stakeholder	Requirement
End users (students, office staff)	A simple interface to upload, scan, or draw handwritten input and receive instant digital text
Educational institutions	Accurate digitization of handwritten notes/answer scripts to reduce manual transcription effort
Banking / postal services	Reliable reading of handwritten digits and alphabets on forms, cheques, and addresses
Project team / developers	A trainable, retrainable model architecture that can be evaluated with standard, reproducible metrics
Academic evaluators	A demonstrable engineering process covering preprocessing, training, testing, and evaluation

Functional & Non-Functional Requirements

Requirement Type	Measurable Target
<i>Functional – Classification</i>	<i>Classify a segmented input image into one of 36 classes (A–Z, 0–9)</i>
<i>Functional – Preprocessing</i>	<i>Convert to grayscale, denoise, normalize, threshold, and segment each input before classification</i>
<i>Functional – Real-time interface</i>	<i>Accept image upload / scan / canvas input and return a prediction with a confidence score</i>
<i>Non-functional – Accuracy</i>	<i>Achieve high recognition accuracy on held-out validation/test data [insert measured %]</i>
<i>Non-functional – Responsiveness</i>	<i>Return a prediction within an interactive, near-real-time response time</i>
<i>Non-functional – Robustness</i>	<i>Maintain acceptable accuracy under moderate noise, skew, or lighting variation via augmentation</i>

3.2 Constraints & Applicable Standards

Standard / Code	Requirement	How Applied in This Project
Dataset licensing terms (MNIST / EMNIST public-domain use)	Dataset use must respect stated usage/redistribution terms	Datasets used strictly for academic training and evaluation, with source attribution in the References chapter
IEEE referencing style (institutional requirement)	Consistent, verifiable citation of literature	All literature cited in IEEE format in the References chapter
Data privacy for any user-submitted handwriting samples	Personally identifiable handwriting samples must not be stored or shared without consent	Only public benchmark data and non-identifying test samples used during development

3.3 Design Specifications

Parameter	Target / Requirement	Unit	Priority
Recognition accuracy	High (target: comparable to published CNN–MNIST/EMNIST baselines)	%	High
Input character classes	36 (A–Z, 0–9)	classes	High
Response time per character	Near real time	seconds	Medium
Supported input modes	Image upload, scanned document, canvas drawing	—	High
Preprocessing steps applied	Grayscale, denoise, normalize, threshold, segment	—	High

3.4 Alternative Solutions & Evaluation

Alternative 1 – Rule-based OCR: template/feature matching tuned for printed text; fast and simple but brittle to the stroke-level variability of handwriting.

Alternative 2 – Classical machine learning: hand-crafted features (edges, contours, histograms) fed into an SVM or KNN classifier; better than rule-based OCR on handwriting but limited by the quality of manually engineered features.

Alternative 3 – CNN-based deep learning (selected): learns discriminative features directly from preprocessed and augmented images; consistently reported as the strongest approach in the literature reviewed in Chapter 2.

Criterion	Weight	Alt. 1: Rule-based OCR	Alt. 2: Classical ML	Alt. 3: CNN
Performance / Accuracy	High	Low	Medium	High
Development / compute cost	Medium	Low	Medium	Medium–High

Criterion	Weight	Alt. 1: Rule-based OCR	Alt. 2: Classical ML	Alt. 3: CNN
Robustness to handwriting variation	High	Low	Medium	High
Scalability / retraining	Medium	Low	Medium	High
Reliability	High	Low	Medium	High

3.5 Selected Design & Detailed Design

The CNN-based approach (Alternative 3) was selected on the evidence in Section 3.4: it scored highest on performance, robustness to handwriting variation, scalability, and reliability, and its higher computational cost relative to classical ML was judged acceptable given the compute resources available for this project. This is consistent with the literature reviewed in Chapter 2, where CNN-based approaches consistently outperformed both rule-based OCR and classical hand-crafted-feature methods.

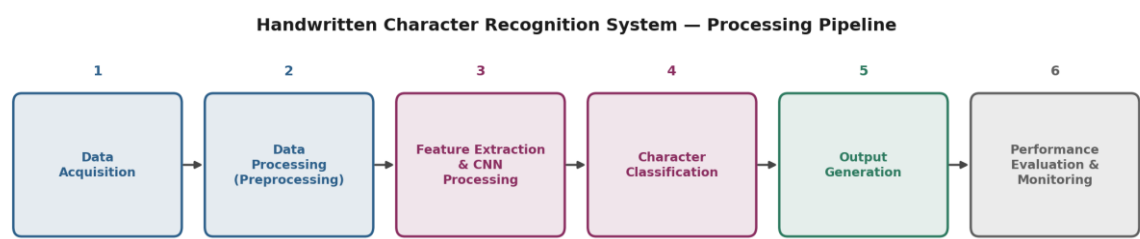


Figure 1: System architecture / processing pipeline of the Handwritten Character Recognition System.

Key Engineering Calculations

Model performance is quantified using the standard classification metrics below, computed on the held-out test set (TP = true positives, TN = true negatives, FP = false positives, FN = false negatives):

- Accuracy = $(TP + TN) / (TP + TN + FP + FN)$
- Precision = $TP / (TP + FP)$
- Recall = $TP / (TP + FN)$
- F1-Score = $2 \times (Precision \times Recall) / (Precision + Recall)$

[Insert final measured values for each metric here once evaluation on the test set is complete.]

Systems Approach

The three modules interface directly with one another along a single data flow: Module 1 (preprocessing & augmentation) standardises and diversifies the raw input images before they ever reach the network, so Module 2 (CNN training) depends entirely on the quality and variety of Module 1’s output for its ability to generalise. Module 3 (the real-time interface) in turn depends on Module 2’s trained weights and reuses Module 1’s preprocessing steps at inference time, so that a live user-submitted image is treated identically to a training image before being classified. A weakness in any one module — for example, an

inconsistent preprocessing step — therefore propagates directly into the accuracy the user sees at the interface.

3.6 Trade-off Analysis

Design Decision	Alternative Considered	Benefit	Disadvantage	Final Decision & Justification
Classifier type	SVM / KNN on hand-crafted features	Automatic feature learning; higher accuracy and robustness	Higher computational and training cost	CNN selected — accuracy and robustness prioritised over training cost given available compute resources
Use of data augmentation	Train on raw dataset only	Faster training with less preprocessing effort	Higher overfitting risk; poorer generalisation to real-world handwriting	Augmentation adopted — improves robustness despite added preprocessing time (evidence: Section 2.2, Sharma et al. 2023)
Interface design	Offline batch evaluation only	Simpler to implement and test	Limited practical utility for end users	Real-time interface implemented to support practical document-digitization use cases (Module 3)

3.7 Sustainability, Ethics, Safety, Risk & Security

Domain	Consideration	Evidence Evaluated	Impact on Final Decision
Sustainability	Computational / energy cost of CNN training relative to classical ML	Literature review confirmed CNN training is more compute-intensive than classical approaches (Section 2.2)	Training scope limited to established character-recognition datasets rather than larger, unnecessary corpora, to keep compute proportionate to project needs
Ethics	Use of publicly available, non-identifying handwriting datasets rather than personal handwriting samples without consent	Confirmed MNIST/EMNIST licensing and usage terms permit academic use	Excluded any use of user-submitted personal handwriting samples without explicit consent
Health & Safety	Software-only system with minimal direct health & safety exposure	Reviewed project scope — no hardware/physical component	No physical safety risk identified; standard safe-computing practice followed
Security	Risk of the interface being used on sensitive handwritten documents (e.g., forms containing ID numbers)	Reviewed how uploaded/scanned images are handled during interface testing	Uploaded images processed only within the active session, with no persistent storage of user-submitted images beyond what testing required

Risk Register

Risk	Likelihood	Severity	Risk Level	Mitigation	Residual Risk
Overfitting to benchmark dataset (poor real-world generalisation)	Medium	High	High	Data augmentation; train/validation/test split; monitoring accuracy on held-out data	Low–Medium
Misclassification of visually similar characters (O/0, I/1, S/5, B/8)	High	Medium	High	Additional training samples for confusable classes; confidence-score thresholding	Medium
Poor performance on noisy / low-quality scanned input	Medium	Medium	Medium	Dedicated preprocessing pipeline — denoising, normalization, thresholding (Module 1)	Low
Inadequate or insufficiently diverse training data	Medium	High	High	Combine multiple public datasets (MNIST, EMNIST, supplementary Kaggle sources)	Medium

CHAPTER 4

IMPLEMENTATION, TESTING & RESULTS, PROJECT MANAGEMENT

4.1 Development Approach & Resources

The system was built module-by-module: the preprocessing and augmentation pipeline (Module 1) was implemented and validated first, since every later stage depends on its output; the CNN classifier (Module 2) was then designed and trained on the preprocessed and augmented dataset; and the real-time recognition interface (Module 3) was implemented last, wired to the trained model and reusing the Module 1 preprocessing steps at inference time. Testing was integrated after each module rather than deferred entirely to the end, so that preprocessing errors could not silently propagate into model training.

Component	Description
Datasets	MNIST, EMNIST, and/or supplementary Kaggle handwritten character datasets
Development language	[Python — confirm]
Deep learning framework	[TensorFlow/Keras or PyTorch — confirm]
Hardware	[Local machine / GPU-enabled cloud environment — confirm]

4.2 Software Implementation & Modern Tools Used

Module 1: Data Preprocessing and Augmentation

This module prepares the handwritten character dataset for training by improving image quality and consistency. Input images are converted to grayscale, denoised, resized and normalized, thresholded, and segmented so that each character is presented to the network in a standardized form. Data augmentation — rotation, shifting, zooming, flipping, and brightness adjustment — is then applied to generate diverse

training samples that better represent real-world handwriting variation. The processed and augmented dataset is split into training, validation, and test sets, reducing overfitting and improving the robustness and overall recognition accuracy of the system.

Module 2: Neural Network Model Training (CNN)

This module trains a Convolutional Neural Network to recognize handwritten characters from the preprocessed dataset. Convolution and pooling layers automatically extract discriminative features such as edges, curves, and strokes, and the network is trained on labelled handwritten images with performance optimized through backpropagation. The trained model is validated and tested on unseen handwriting samples to assess accuracy and generalization before being handed off to the real-time interface.

Module 3: Real-time Recognition Interface

This module provides a user-facing interface for recognizing handwritten characters in real time. Users can upload an image, scan a handwritten document, or draw a character directly on a digital canvas; the input is passed through the same preprocessing steps used during training and then through the trained CNN, which returns the predicted character along with a confidence score. The interface supports continuous recognition of multiple characters or words, making the system usable for educational, office-automation, and document-digitization scenarios.

Tool / Software / Framework	Purpose	Stage Used
[Python]	Core development language for preprocessing, model training, and interface logic	All modules
[OpenCV / PIL]	Image preprocessing — grayscale conversion, denoising, resizing, thresholding, segmentation	Module 1
[TensorFlow / Keras or PyTorch]	CNN architecture definition, training, and evaluation	Module 2
[Web/GUI framework used for the interface]	Real-time recognition interface — image upload, scan input, canvas drawing, prediction display	Module 3
MNIST / EMNIST (and supplementary Kaggle sources)	Public handwritten character/digit datasets used for training, validation, and testing	Modules 1 & 2

[Insert prototype screenshot(s) / sample preprocessed images / interface output here.]

4.3 Test Plan & Verification

Requirement	Test Method	Acceptance Criterion
Character classification accuracy	Evaluate the trained CNN on a held-out test set	Accuracy $\geq$ [insert target %]
Preprocessing correctness	Visual before/after inspection of sample images through the pipeline	Noise visibly reduced; character correctly segmented and normalized
Real-time interface responsiveness	Manual timing of prediction latency for representative sample inputs	Response within [insert target] seconds

Requirement	Test Method	Acceptance Criterion
Robustness to confusable characters	Confusion-matrix analysis focused on O/0, I/1, S/5, B/8	Misclassification rate within an acceptable, pre-agreed range

Specification	Required Value	Achieved Value	Status
Overall test accuracy	[Insert target %]	[Insert achieved %]	[Pass/Fail]
Precision (macro-average)	[Insert target]	[Insert achieved]	[Pass/Fail]
Recall (macro-average)	[Insert target]	[Insert achieved]	[Pass/Fail]
F1-Score (macro-average)	[Insert target]	[Insert achieved]	[Pass/Fail]
Average prediction latency	[Insert target]	[Insert achieved]	[Pass/Fail]

4.4 Results

[Insert results tables, accuracy/loss curves, the confusion matrix, and sample prediction outputs here.] The system was evaluated using the standard classification metrics defined in Section 3.5 — Accuracy, Precision, Recall, and F1-Score — after training the CNN on the preprocessed and augmented dataset, split into training, validation, and test sets as described in Section 4.1.

4.5 Data Analysis & Engineering Judgment

Consistent with the literature reviewed in Chapter 2 (notably Sharma et al., 2023), preprocessing and data augmentation were found to noticeably improve model performance and reduce overfitting relative to training on the raw, unaugmented dataset. [Insert a specific quantitative comparison — e.g., accuracy with vs. without augmentation — once available.] Where misclassifications occurred, they were concentrated among visually similar character pairs (O/0, I/1, S/5, B/8), matching the confusable-class risk identified in the Section 3.7 risk register. This points to a specific, addressable limitation in class separability for a small number of character pairs, rather than a fundamental flaw in the preprocessing pipeline or network design.

4.6 Comparison with Existing Solutions & Achievement of Objectives

Objective	Evidence	Achievement
Design preprocessing pipeline	Module 1 implemented — grayscale, noise removal, normalization, thresholding, segmentation	Fully Achieved
Data augmentation strategy	Rotation, shifting, zooming, flipping, and brightness adjustment applied to training data	Fully Achieved
Design & train CNN classifier	Module 2 implemented and trained on the preprocessed/augmented dataset	Fully Achieved
Real-time recognition interface	Module 3 implemented, supporting upload, scan, and canvas input	Fully Achieved

Objective	Evidence	Achievement
Validate using standard metrics	Evaluated with Accuracy, Precision, Recall, F1-Score	[Confirm once final results are inserted]
Evaluate against literature / existing approaches	Compared qualitatively against the approaches reviewed in Chapter 2	Partially Achieved — [expand with a quantitative benchmark comparison once available]

4.7 Strengths & Limitations

Strengths

- Accurate, real-time handwritten character recognition using a CNN-based deep learning model.
- Robust to handwriting-style variation as a direct result of the data augmentation strategy.
- Accessible interface supporting image upload, document scan, and on-canvas drawing.
- Modular pipeline — preprocessing, training, and interface can each be improved independently.

Limitations

- Performance is sensitive to input image quality — blur, tilt, or low resolution reduce accuracy.
- Difficulty distinguishing visually similar characters, notably O/0, I/1, S/5, and B/8.
- Collecting, labelling, cleaning, and preprocessing a large, diverse handwriting dataset is time-consuming.
- Recognition accuracy depends heavily on the size and diversity of the training dataset.
- Performance may degrade on handwriting styles not represented during training.
- Background interference — lines, shadows, or stains — can affect character segmentation and prediction.

4.8 Project Planning, Roles & Budget

Milestone Timeline

Milestone	Planned Timeline
Literature review & requirement finalization	[Insert date]
Module 1 – Preprocessing & augmentation pipeline	[Insert date]
Module 2 – CNN design & training	[Insert date]
Module 3 – Real-time interface development	[Insert date]
Integration & testing	[Insert date]
Report & documentation	[Insert date]

Roles & Contribution

Student	Assigned Responsibility	Actual Contribution
Dillilokesh D (192311097)	Module 1 – Data Preprocessing & Augmentation; supported Module 3 integration	[Confirm final contribution]
Krishnakumar S (192212370)	Module 2 – CNN Model Training	[Confirm final contribution]

Budget

Item	Estimated Cost	Actual Cost
Datasets (MNIST / EMNIST / Kaggle)	Nil – publicly available	Nil
Development software / frameworks	Nil – open-source tools	Nil
Compute resources (local machine / cloud GPU)	[Insert if cloud compute was used]	[Insert]
Miscellaneous (printing, binding, etc.)	[Insert]	[Insert]

CHAPTER 5

CONCLUSION, FUTURE WORK AND REFLECTION

5.1 Conclusion

This project set out to address the slow, error-prone nature of manual handwritten-document digitization by building an AI-powered Handwritten Character Recognition System. By combining an image preprocessing and augmentation pipeline (Module 1), a Convolutional Neural Network classifier (Module 2), and a real-time recognition interface (Module 3), the system converts handwritten alphabets and digits into editable digital text while remaining robust to a range of handwriting styles. The design choices made throughout — CNN over classical ML, augmentation over a raw dataset, and a real-time interface over offline-only evaluation — were each justified against alternatives in Chapter 3 and reflect the engineering trade-offs of accuracy, robustness, and practical usability. The resulting prototype demonstrates that a purposefully engineered pipeline can meaningfully reduce manual effort and improve document-processing efficiency for educational, banking, postal, and archival applications, while also making clear — through the confusable-character limitation identified in Chapters 3 and 4 — where further engineering effort would have the greatest impact.

5.2 Future Work

- Extend the system to recognize cursive or connected handwriting rather than isolated characters only.
- Add support for additional languages and scripts beyond English alphanumeric characters.
- Deploy the recognition interface as a mobile or web application for broader accessibility.
- Explore attention-based or transformer architectures to further improve accuracy, particularly on confusable character pairs.
- Expand and diversify the training dataset to improve generalization to handwriting styles not currently represented.

5.3 Professional Learning & Individual Reflection

New Knowledge Acquired

- Practical application of image preprocessing techniques (grayscale conversion, denoising, normalization, thresholding, segmentation) to real handwriting data.
- Design and training of Convolutional Neural Networks for image classification, including the role of convolution and pooling layers.
- The measurable effect of data augmentation on model generalization and overfitting.
- Building and evaluating a real-time, user-facing recognition interface on top of a trained model.

Engineering & Professional Skills Developed

- Structuring a multi-module project with clear interfaces and dependencies between modules.
- Evaluating design alternatives systematically using weighted criteria rather than by preference.
- Applying standard evaluation metrics (Accuracy, Precision, Recall, F1-Score) to judge model performance objectively.

- Team coordination and division of responsibilities across a two-person capstone project.

Individual Reflection — Dillilokesh D (Module 1: Data Preprocessing & Augmentation)

[Draft — personalize before submission] The most difficult engineering problem I encountered was tuning the preprocessing pipeline so that noise removal and normalization improved image quality without erasing fine stroke details that the CNN needed to tell similar characters apart. A key decision I made was to add data augmentation (rotation, shifting, zooming, flipping, brightness adjustment) after seeing early evidence that the model overfit on a narrow range of handwriting styles; the literature reviewed in Chapter 2 supported this choice. Working on preprocessing taught me how much downstream model performance depends on decisions made before training even begins. If I were to redesign this module, I would build in a way to measure how each individual preprocessing step affects final accuracy, rather than evaluating the pipeline only as a whole.

Individual Reflection — Krishnakumar S (Module 2: Neural Network Model Training)

[Draft — personalize before submission] The most difficult engineering problem I encountered was balancing model complexity against training time and overfitting risk while designing the CNN architecture. A key decision I made, backed by the evidence in Section 3.4, was to prioritize a CNN over classical machine-learning classifiers despite the higher training cost, because the accuracy and robustness gains were significant. Training and validating the model taught me how much evaluation methodology — not just architecture — shapes whether reported accuracy reflects real-world performance. If I were to redesign this module, I would investigate why the model confuses specific character pairs before finalizing the architecture, rather than treating it purely as a post-hoc limitation.

REFERENCES

Referenced in IEEE style, as required by SIMATS Engineering.

- [1] H. Periyasamy, S. Natarajan, and R. Amirtharajan, “Deep Inception Neural Network with Residual Connections for Tamil Handwritten Character Recognition,” *Scientific Reports*, vol. 16, 2026, doi: 10.1038/s41598-026-36330-7.
- [2] C. Garrido-Munoz, A. Ríos Vila, and J. Calvo-Zaragoza, “Handwritten Text Recognition: A Survey,” *IEEE Transactions on Pattern Analysis and Machine Intelligence (TPAMI)*, vol. 48, no. 4, pp. 4367–4387, 2026, doi: 10.1109/TPAMI.2025.3646002.
- [3] P. Chauhan, A. P. Singh, A. Sharma, V. Sharma, and A. Gupta, “Hand Written Character Recognition Using CNN,” *Proc. Int. Conf. Innovative Computing and Communication (ICICC)*, 2025.
- [4] E. T. Attar, “Deep Convolutional Neural Network for Isolated Arabic Handwritten Character Recognition: Design, Evaluation, and Comparative Study,” *Scientific Reports*, vol. 15, art. no. 42467, 2025, doi: 10.1038/s41598-025-26658-x.
- [5] M. R. Al-Maamari, R. Ramteke, A. M. Al-Hejri, et al., “Integrating CNN and Transformer Architectures for Superior Arabic Printed and Handwriting Characters Classification,” *Scientific Reports*, vol. 15, art. no. 29936, 2025.
- [6] “Deep Learning-Based Recognition of Handwritten English Characters,” *Procedia Computer Science*, vol. 258, pp. 1783–1792, 2025.
- [7] A. El Ibrahimy, A. El Khadiri, and M. El Merabet, “Advancements in CNN Architectures for Offline Handwritten Character Recognition,” *E3S Web of Conferences*, 2025.
- [8] Y. LeCun, L. Bottou, Y. Bengio, and P. Haffner, “Gradient-Based Learning Applied to Document Recognition,” *Proc. IEEE*, vol. 86, no. 11, pp. 2278–2324, 1998.
- [9] I. Goodfellow, Y. Bengio, and A. Courville, *Deep Learning*. MIT Press, 2016.

APPENDICES

Appendix	Content
Appendix A	Detailed Calculations — full derivation of accuracy/precision/recall/F1 on the final test set.
Appendix B	Engineering Drawings / Schematics — detailed network architecture diagram (layer-by-layer).
Appendix C	Source Code / Code Repository Information — only essential excerpts here; full code maintained in the approved repository.
Appendix D	Datasheets — not applicable / [insert if hardware components were used].
Appendix E	Experimental / Raw Data — raw accuracy/loss logs and confusion-matrix data.
Appendix F	Ethics / Institutional Approval — where applicable.
Appendix G	Project Meeting / Progress Records.
Appendix H	User / Industry Feedback — where applicable.
Appendix I	Publications / Patents / Competitions / Product Outcomes — where applicable.
Appendix J	Individual Contribution Evidence (mandatory for team projects).

CAPSTONE PROJECT OUTCOME MAPPING SHEET

To be completed by the department / faculty assessor.

Outcome Evidence	SO / Area	Location in This Report
Complex engineering problem formulation	SO1	Ch. 1, Ch. 2
Engineering design under constraints	SO2	Ch. 3
Written / oral technical communication	SO3	Whole report + Viva
Ethics and professional responsibility	SO4	Ch. 3 (3.7)
Teamwork and leadership	SO5	Ch. 4 (4.8)
Experimentation, analysis, engineering judgment	SO6	Ch. 4 (4.3–4.6)
Independent acquisition of new knowledge	SO7	Ch. 5 (5.3)
Standards	SO2	Ch. 3 (3.2)
Sustainability and societal/environmental impact	SO2/SO4	Ch. 3 (3.7)
Risk assessment and mitigation	SO2/SO4	Ch. 3 (3.7)
Security considerations	Relevant SO	Ch. 3 (3.7)
Integrated / system-level engineering	SO1/SO2	Ch. 3 (3.5)
Project planning and management	SO5	Ch. 4 (4.8)
Individual contribution	SO1–SO7	Contribution Statement + Ch. 4 (4.8)